WEEK 1 · LESSON 2 OF 24 · HANDOUT

Tokens: How LLMs Read Text

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=zduSFxRajkE — open the lesson online to watch inside the course.

🎓 **Go deeper:** Andrej Karpathy: Let's build the GPT Tokenizer — https://www.youtube.com/watch?v=zduSFxRajkE

🎓 **Go deeper:** Andrej Karpathy: Deep Dive into LLMs like ChatGPT — https://www.youtube.com/watch?v=7xTGNNLPyMI

Your goal in this lesson: Explain what tokens are, build a byte-pair-encoding (BPE) tokenizer from scratch, and explain why token counts matter for cost, context limits and fairness across languages.

LLMs don't see words — they see tokens

Before an LLM reads anything, a **tokenizer** cuts the text into **tokens**: common words become one token, rarer words are split into pieces. "unbelievable" might become "un", "believ", "able". Each token has an id number, and the model only ever sees those numbers.

Why not whole words? There are too many (every name, typo and language). Why not single letters? Sequences become very long and slow. **Sub-word tokens** are the compromise.

Rough rule of thumb for English: **1 token ≈ ¾ of a word**, or about 4 characters.

Byte-pair encoding (BPE)

Most LLM tokenizers use BPE:

1. Start with single characters.
2. Find the **most frequent pair** of neighbouring tokens in the training text.
3. **Merge** that pair into a new token. Repeat thousands of times.

Frequent words end up as single tokens; rare words stay split.

```
import random

random.seed(0)
names = ["Aline", "Eric", "Grace", "Jean", "Diane", "Patrick", "Claudine", "Samuel", "Ange",
"Didier"]
places = ["Kigali", "Huye", "Musanze", "Rubavu", "Nyagatare", "Rusizi", "Muhanga"]
goods = ["tomatoes", "beans", "bananas", "avocados", "maize", "potatoes", "milk", "coffee",
"rice", "cassava"]
verbs = ["sells", "buys", "carries", "cooks", "packs", "counts"]
times = ["in the morning", "at noon", "in the evening", "on Saturday", "after the rain",
"before school"]
moods = ["happy", "tired", "busy", "proud", "calm", "hungry"]
templates = [
    "{n} {v} {g} in {p} {t}.",
    "In {p}, {n} {v} {g} {t}.",
    "{n} is {f} because the {g} are fresh.",
    "{t}, {n} goes to the market in {p}.",
    "{n} pays for the {g} with mobile money.",
    "The price of {g} in {p} is low {t}.",
    "{n} tells {m} that the {g} from {p} are the best.",
    "{n} and {m} share {g} {t}.",
]

def sentence():
    n, m = random.sample(names, 2)
    s = random.choice(templates).format(n=n, m=m, v=random.choice(verbs),
g=random.choice(goods),
                                        p=random.choice(places), t=random.choice(times),
f=random.choice(moods))
    return s[0].upper() + s[1:]

text = "\n".join(sentence() for _ in range(4000))
print(len(text), "characters")
print(text[:250])
```

Build BPE from scratch

```
from collections import Counter

def get_pairs(tokens):
    return Counter(zip(tokens, tokens[1:]))

def merge(tokens, pair, new_token):
    out, i = [], 0
    while i < len(tokens):
        if i < len(tokens) - 1 and (tokens[i], tokens[i + 1]) == pair:
            out.append(new_token)
            i += 2
        else:
            out.append(tokens[i])
            i += 1
    return out

def train_bpe(training_text, n_merges):
    tokens = list(training_text)
    merges = []
    for _ in range(n_merges):
        pairs = get_pairs(tokens)
        if not pairs:
            break
        best = pairs.most_common(1)[0][0]
        merges.append(best)
        tokens = merge(tokens, best, best[0] + best[1])
    return merges

merges = train_bpe(text[:20000], n_merges=150)
print("first merges:", merges[:8])
print("later merges:", merges[-5:])
```

The first merges join very common pairs such as "e" + " " and "s" + " "; later merges build whole words and phrases such as "the ", "Eric" and even "after the rain".

Encode new text

```
def encode_bpe(s, merges):
    tokens = list(s)
    for pair in merges:                       # apply merges in the order they were learned
        tokens = merge(tokens, pair, pair[0] + pair[1])
    return tokens

sentence = "Grace sells avocados in Kigali in the morning."
tokens = encode_bpe(sentence, merges)
print(tokens)
print(len(sentence), "characters ->", len(tokens), "tokens")
```

The 46-character sentence becomes about 21 tokens: frequent pieces have become single tokens, while the rarer name "Kigali" is still split into small parts.

The language gap

Our tokenizer learned only from English market sentences. Watch what happens with text it has not seen much of:

```
samples = {
    "English": "Grace pays for the beans with mobile money.",
    "Kinyarwanda": "Grace yishyura ibishyimbo akoresheje mobile money.",
    "Made-up word": "Xqzvbrtwplk",
}
for language, s in samples.items():
    t = encode_bpe(s, merges)
    print(f"{language:12s} {len(s):3d} characters -> {len(t):3d} tokens   ({len(s) /
len(t):.1f} characters per token)")
```

The English sentence becomes only **8** tokens (more than 5 characters each). The same meaning in Kinyarwanda needs about **36** tokens — the tokenizer barely knows the language, so it falls back to tiny pieces — and the random word is cut into single letters. Real tokenizers show the same pattern: text in languages that were rare in the tokenizer's training data — including many African languages — needs **more tokens** for the same meaning. That means:

- **Higher cost** — APIs charge per token.
- **Less fits in the context window** — the model "sees" less of your document.
- Often **lower quality**, because the model saw less of the language.

Newer models are improving their coverage of African languages, and projects across Africa are building better data for them.

Tokens decide cost and limits

- **Pricing:** LLM APIs charge per million input tokens and per million output tokens (output usually costs more).
- **Context window:** the maximum number of tokens the model can consider at once — from a few thousand in early models to hundreds of thousands or more today. Your prompt, documents, conversation history **and** the answer must all fit.
- **Spelling and counting:** because the model sees tokens, not letters, tasks like "how many r's are in strawberry" or reversing a word can trip it up.

Key terms

- **Token** — a piece of text the model reads as one unit, with an id number.
- **Tokenizer** — the tool that splits text into tokens and back.
- **BPE** — byte-pair encoding: repeatedly merge the most frequent pair.
- **Vocabulary** — all the tokens a tokenizer knows (often 50,000–200,000).
- **Context window** — the most tokens a model can take in at once.

Remember this: LLMs read sub-word tokens; how a language is tokenized affects price, how much text fits, and quality — something to watch when building for Kinyarwanda and other African languages.

Name: _____ Date: _____

Activity

Task 1

Train BPE with 50, 150 and 400 merges. How does the token count of the English sentence change?

Task 2

Add 20 Kinyarwanda sentences to the training text and retrain. Does the Kinyarwanda token count drop?

Task 3

Write a `decode_bpe` function that joins tokens back into text. Is it always perfect?

Task 4

Paste the same paragraph in English and in Kinyarwanda into an online LLM tokenizer tool and compare the counts.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does a tokenizer do?

   A. Checks grammar
   B. Translates text
   C. Splits text into tokens with id numbers
   D. Generates answers

2. How does BPE build its vocabulary?

   A. At random
   B. By sorting words
   C. By repeatedly merging the most frequent pair of tokens
   D. By using a dictionary

3. Roughly how many characters is one English token?

   A. About 20
   B. 100
   C. 1
   D. About 4

4. Why can text in under-represented languages cost more with LLM APIs?

   A. It uses images
   B. It is split into more tokens, and APIs charge per token
   C. Those languages are banned
   D. It is longer to type

5. What is the context window?

   A. The maximum number of tokens the model can take in at once
   B. The number of users
   C. The chat app's window size
   D. The training time

6. Why might an LLM miscount the letters in a word?

   A. It cannot count at all
   B. It has no vocabulary
   C. Letters are hidden
   D. It sees tokens, not individual letters

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.